



## Trabajo Práctico Final Sistemas Digitales 2



### Diseño e implementación del juego del dinosaurio de Google Chrome en FPGA Spartan 6

Victor Curiel, Ximena Quenhan

Facultad de Ingeniería UNA, Ing. Mecatrónica

#### PALABRAS CLAVE

MIPS, FPGA, VHDL, Ensamblador, Juego 2D

#### RESUMEN

Este trabajo estudia el lenguaje ensamblador para el procesador MIPS, implementado en una FPGA Spartan-6 (Mimas V2), utilizando VHDL para su descripción y ejecución de instrucciones desarrolladas en el simulador MARS, con el objetivo de simular un juego del dinosaurio. Para ello, se aplican técnicas de programación tanto en VHDL como en ensamblador.

#### 1. Introducción

Los videojuegos en hardware reconfigurable se han convertido en una excelente herramienta para enseñar y aplicar conceptos de diseño digital. En este proyecto se optó por implementar una versión del conocido juego del dinosaurio de Google Chrome, que aparece cuando el navegador no tiene conexión a Internet. Este juego es simple pero lo suficientemente interesante como para poner en práctica conceptos como controladores VGA, temporizadores, detección de colisiones y máquinas de estados.

#### 2. Justificación

La elección del juego del dinosaurio se basa en su baja complejidad gráfica, lo que lo hace ideal para plataformas con recursos limitados como la FPGA Spartan-6. Además, la lógica detrás del juego permite aplicar y reforzar conocimientos fundamentales de diseño digital como temporización, FSMs, y manejo de gráficos en tiempo real.

A nivel de implementación física, se planteó llevar el juego al entorno de hardware utilizando una placa FPGA Spartan-6. Esto permite evaluar el rendimiento del juego ejecutado en una arquitectura real y visualizar su salida a través de la interfaz VGA.

#### 3. Diseño del algoritmo

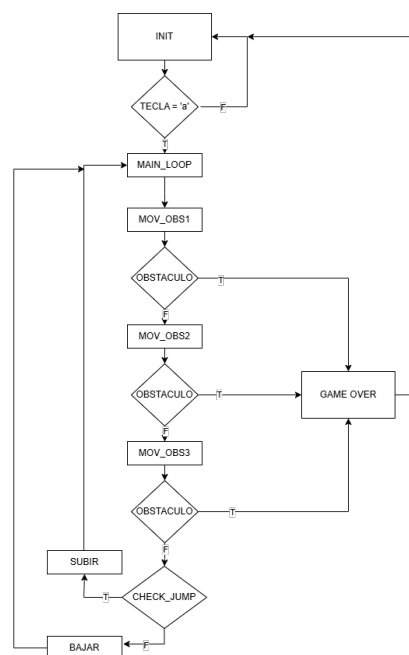


Fig. 2. ASM del juego

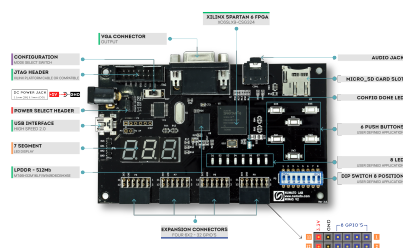


Fig. 1. Placa de desarrollo Mimas V2 S6 FPGA

El juego del dinosaurio comienza siempre en la rutina de inicialización, donde se establecen las posiciones iniciales de los tres obstáculos y del dinosaurio, se pinta el fondo y el borde, y se dibujan las entidades en pantalla. A partir de aquí, el comportamiento se articula en un bucle principal que recorre, uno a uno, los tres estados de movimiento de obstáculos: en cada uno se repinta la nueva posición del obstáculo y se comprueba colisión.

Entre las acciones disponibles, la única interacción del jugador es el salto ('checkjump'). La acción por defecto es

el descenso, es decir, el dinosaurio baja lentamente si no hay pulsación. Cuando el jugador presiona 'a', la máquina verifica que el personaje no exceda el límite superior antes de ejecutar la rutina de salto; después, espera a que se suelte la tecla para evitar saltos continuos. Si en cualquier punto—tanto durante el movimiento de un obstáculo como tras un salto o descenso—se detecta una colisión, se transita al estado Game Over. Finalmente, al pulsar de nuevo 'a', el juego se reinicia desde la rutina de inicialización y el ciclo se repite indefinidamente.

#### 4. Hardware del juego

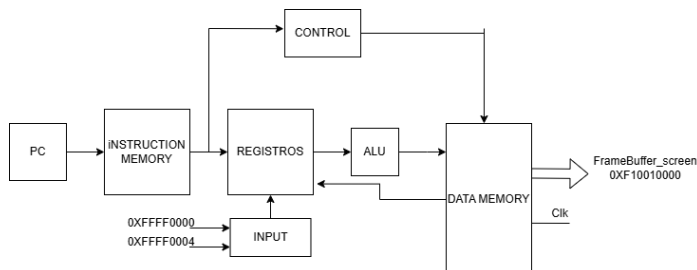


Fig. 3. DataPath del MIPS

- **Program Counter:** Apunta a la dirección de memoria que se va a ejecutar.
- **Instruction Memory:** Recibe la dirección suministrada por el Program Counter (PC) para devolver la instrucción correspondiente.
- **Registros:** Registros internos que se encargan de almacenar temporalmente los datos.
- **Input Control (Control de entrada):** Recibe las señales externas necesarias para el juego, en este caso detecta la tecla de salto y envía esa información a los registros para su uso.
- **ALU (Arithmetic Logic Unit):** Realiza operaciones básicas como sumas, comparaciones o restas según sea necesario para el programa.
- **Data Memory (Memoria de datos):** Se encarga de los datos dinámicos del programa, como diferentes variables o estados del juego.
- **Control Unit (Unidad de Control):** Es el encargado de generar las señales correspondientes para el correcto funcionamiento del juego según la instrucción dada por el PC.
- **FrameBuffer screen:** Es la dirección inicial donde el procesador MIPS define los valores de color de cada píxel, sirviendo como un lienzo en memoria que conserva la imagen completa antes de enviarla al monitor.

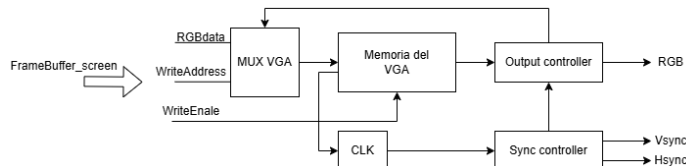


Fig. 4. DataPath del VGA

- **RGB Data:** Valor RGB (Red-Green-Blue) del color que se quiere escribir en la pantalla.
- **Write Address:** La dirección de memoria en la que debe escribirse el color.
- **WriteEnable:** Es el indicador que avisa al procesador si debe o no escribir datos en la memoria VGA, para poder actualizar el juego.
- **CLK (Reloj):** Reloj de sincronización para toda actividad y proceso del sistema, así todas las operaciones que hay entre el procesador y la salida sean coordinadas.
- **MUX VGA:** Multiplexor que decide si se accede a la escritura del procesador o a la lectura del controlador VGA y dependiendo de eso se mandan los datos a la memoria VGA.
- **Memoria VGA (VRAM):** Almacena los píxeles que deben mostrarse en pantalla con los colores correspondientes a utilizar.
- **Controlador de salida VGA (Output Controller):** Desde la VRAM selecciona los datos RGB a ser enviados al monitor.
- **Controlador de sincronización (Sync Controller):** Envía las señales Hsync y Vsync que se encargan de las líneas de píxeles horizontales y verticales específicamente y las sincroniza para evitar errores en la visualización del monitor.

#### 5. Descripción general del juego

##### 5.1. Componentes del juego

El juego está compuesto por los siguientes elementos principales:

- **Jugador (Dinosaurio):** Representado como un píxel pintado en color verde oscuro. Se ubica en una posición fija y puede saltar hacia arriba y volver a caer.
- **Obstáculos:** Existen tres obstáculos que se mueven de derecha a izquierda en diferentes patrones: algunos de forma horizontal y otros de forma vertical.
- **Pantalla:** Se representa mediante un arreglo de 768 bytes, simulando una grilla de memoria donde cada dirección representa un "píxel" de 4 bytes. La pantalla contiene el fondo, bordes, jugador y los obstáculos, que se actualizan en cada ciclo de juego.

##### 5.2. Dinámica del juego

El usuario inicia el juego con una tecla. Desde ese momento, el jugador puede realizar saltos con la tecla a. Si no se detecta una entrada, el jugador caerá automáticamente por gravedad simulada. Mientras tanto, los obstáculos se desplazan de derecha a izquierda, creando la ilusión de movimiento.

## 6. Algoritmo y lógica de juego

### 6.1. Inicialización

- Se cargan colores de fondo y bordes mediante las funciones `llenar_pantalla` y `borde`.
- Se inicializan las posiciones de los tres obstáculos con los registros `$s5`, `$s6` y `$s7`, y del jugador con `$s0`.

```

1 inicio:
2 ori $s5, $0, 344
3 ori $s6, $0, 360
4 ori $s7, $0, 376
5 ori $s0, $0, 388
6 ori $a0, $0, 0x29dfff
7 jal llenar_pantalla
8 ori $a0, $0, 0xf9ff37
9 jal borde

```

Este fragmento establece las posiciones iniciales de cada obstáculo y del jugador, y pinta el fondo y los bordes del escenario del juego.

### 6.2. Bucle principal

- Se llama a las rutinas `mov_obs1`, `mov_obs2` y `mov_obs3` para mover los obstáculos.
- Se verifica la posición del jugador: si está por encima del piso, se llama a `bajar`.
- Si se detecta la tecla `a` (ASCII 97), se invoca la función `subir`, que mueve al jugador 192 bytes arriba.
- Cada movimiento de obstáculo llama a `colision`, que evalúa si el jugador tocó algún obstáculo.

```

1 main_loop:
2 j mov_obs1
3
4 vuelta1:
5 j mov_obs2
6
7 vuelta2:
8 j mov_obs3
9
10 vuelta3:
11 slti $s1, $s0, 388
12 bne $s1, 1, main
13 j bajar

```

Este bloque representa la secuencia principal del juego, donde se alternan los movimientos de los obstáculos y se actualiza el estado del jugador en cada iteración.

### 6.3. Simulación de gravedad

La gravedad se implementa de forma simple comparando la posición actual del jugador con el suelo. Si el jugador está en el aire, se ejecuta la función `bajar`, que incrementa su dirección en memoria (equivalente a moverlo hacia abajo en pantalla).

## 6.4. Lógica de movimiento de obstáculos

### Movimiento de Obstáculo:

```

1 mov_obs1:
2 ori $a0, $0, 0x29dfff
3 jal obstaculo1
4 addi $s5, $s5, -4
5 beq $s5, 316, derecha1
6 j salto1
7
8 derecha1:
9 ori $s5, $0, 380
10
11 salto1:
12 ori $a0, $0, 0xffffffff
13 jal obstaculo1
14 jal colision
15 ori $a0, $0, 40000
16 jal esperar
17 j vuelta1

```

Este bloque de código representa la lógica general para el movimiento de un obstáculo horizontal en pantalla. Se realiza en los siguientes pasos:

- Primero, se borra el obstáculo actual dibujándolo con el color del fondo (líneas 2 y 3).
- Luego, se actualiza su posición restando 4 bytes, lo cual equivale a desplazarlo 1 columna hacia la izquierda en la memoria (línea 4).
- Si el obstáculo alcanza el borde izquierdo (posición 316), se reinicia su valor a una posición más a la derecha (líneas 5 y 6).
- A continuación, se vuelve a dibujar el obstáculo con su color original (líneas 7 y 8).
- Se llama a la rutina de colisión para verificar si el jugador colisionó con el obstáculo (línea 9).
- Finalmente, se aplica una pausa con `esperar` antes de continuar con la siguiente iteración (líneas 10 y 11).

Dado que todos los obstáculos comparten la misma lógica de movimiento, esta descripción es aplicable a los tres casos (`mov_obs1`, `mov_obs2` y `mov_obs3`) simplemente cambiando el registro que representa su posición.

### 6.5. Colisiones

```

1 colision:
2 beq $s0, $s5, game_over
3 addi $t0, $s5, 64
4 beq $s0, $t0, game_over
5 beq $s0, $s6, game_over
6 beq $s0, $s7, game_over
7 addi $t1, $s7, 64
8 beq $s0, $t1, game_over
9 jr $ra

```

La rutina compara la posición actual del jugador con las posiciones de los obstáculos. Si hay coincidencia directa o vertical (+64 en obstáculo 1 y 3), se activa el estado de `game_over`. En el caso del obstáculo 2, al ser más bajo, solo se evalúa colisión en una única dirección horizontal exacta.

[Puedes consultar el código completo en el repositorio.](#)

## 7. Simulación

Se hará la simulación en el MARS usando Bitmap y Keyboard

### 7.0.1. Configuración de los simuladores

Primero se debe realizar la configuración de los simuladores de teclado y display o keyboard y bitmap, para el correcto funcionamiento del código en el MARS, para el teclado solo es necesario desplegar el mismo y conectarlo. En la **Figura 5** se puede apreciar como debe quedar el simulador de teclado.

Para el simulador de display se deben realizar algunas configuraciones, lo primero es cambiar el tamaño por defecto de los bits, de 1 a 32, tanto el ancho como el alto, luego la dirección base para visualización "Base address for display" se debe cambiar de **static data** a **heap**, finalmente es necesario establecer la conexión con el MIPS. En la **Figura 6** se puede apreciar como debe quedar la configuración del simulador de display.

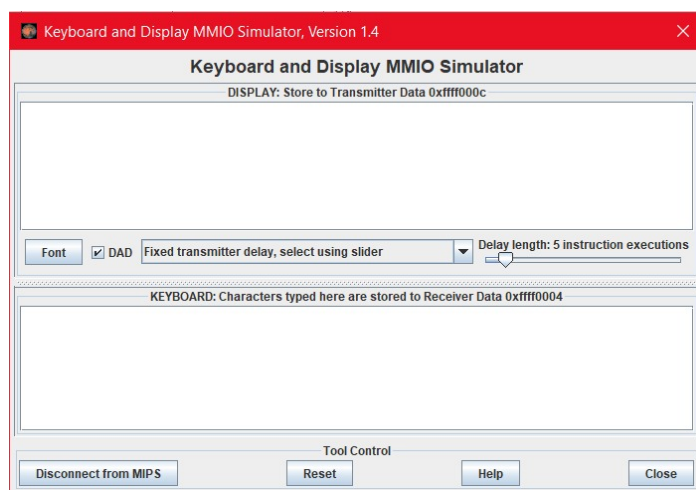


Fig. 5. Keyboard. Simulador de teclado

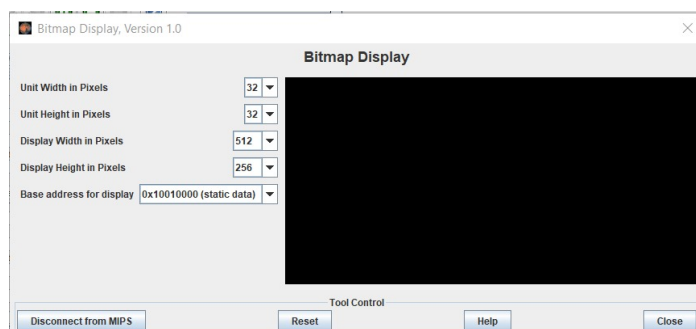


Fig. 6. BitMap Display

### 7.0.2. Prueba de funcionamiento

En la **Figura 7** se observa como el dinosaurio salta al presionar la tecla **a** en el simulador, mientras los bloques se mueve de derecha a izquierda.

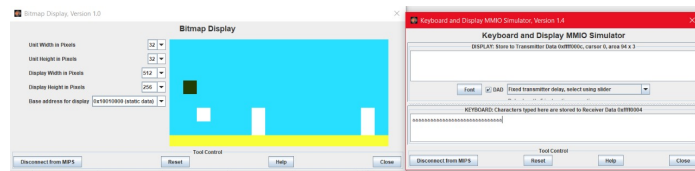


Fig. 7. Dinosaurio saltando

En la **Figura 8** se observa como se pone todo en rojo, lo cual indica el dinosaurio choco y que el juego llego a su fin **GAME OVER**.

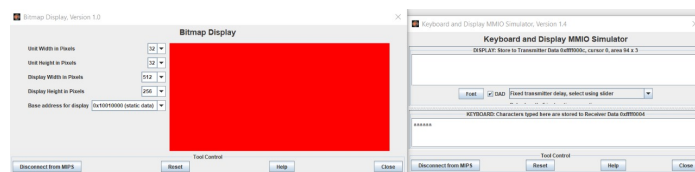


Fig. 8. GameOver. Colisión del dino y final del juego

## 8. Implementación

La implementación del proyecto se llevó a cabo en dos etapas principales: la simulación en el entorno MARS y la prueba final en hardware real, utilizando una FPGA Spartan-6 con salida VGA. Durante el desarrollo, se realizaron numerosas iteraciones, ajustes y pruebas para adaptar el código ensamblador al entorno físico, lo que implicó comprender en profundidad el funcionamiento del hardware involucrado.

En la simulación, se logró representar de forma gráfica el juego completo, incluyendo el fondo, el jugador (dinosaurio), y los obstáculos en movimiento, como se observa en la Figura 7. Esta etapa fue fundamental para validar la lógica del juego, el control por teclado y la manipulación del framebuffer.

Al trasladar el código al hardware, se enfrentaron desafíos naturales de integración entre software y sistema embebido. A pesar de que no se logró visualizar el juego completo en la pantalla VGA, sí se obtuvo una representación visual coherente del entorno, con bloques dibujados correctamente, como se muestra en la Figura 9.

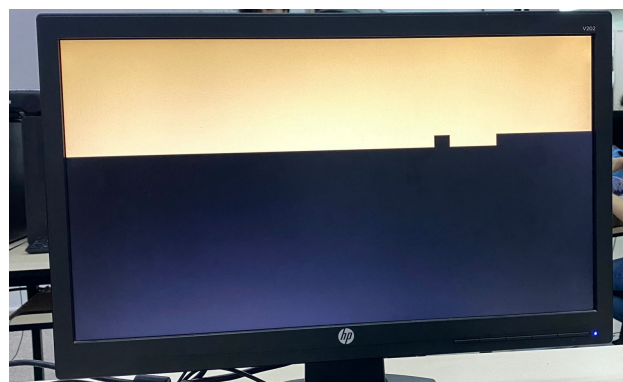


Fig. 9. Resultado en pantalla utilizando salida VGA en FPGA Spartan-6

Este proceso fue altamente formativo, ya que permitió profundizar en conceptos clave como el manejo del framebuffer, la configuración del display VGA, el diagrama de bloques ASM, la estructura del procesador MIPS y su interacción con periféricos mediante memoria mapeada.

Además, se adquirió una visión más clara sobre los límites y posibilidades del diseño digital en sistemas reales.

Aunque el resultado final mostrado en hardware no abarcó todos los elementos gráficos del juego, el proyecto nos ayudó a entender como demostrar la interacción directa entre un procesador MIPS, su codificación en lenguaje ensamblador y la representación visual en una plataforma de hardware digital. El aprendizaje obtenido durante el camino fue significativo y sentó las bases para futuras implementaciones más complejas.

## 9. Conclusión

En conclusión, el proyecto demuestra con éxito la viabilidad de implementar un juego clásico en una plataforma de hardware digital utilizando MIPS y VHDL, destacando los beneficios de una interacción directa entre software y hardware en un sistema de procesamiento en tiempo real. La integración de entradas por teclado, control lógico secuencial y salida gráfica a través de un framebuffer evidencia cómo conceptos fundamentales del diseño digital pueden aplicarse en una experiencia interactiva concreta.

Esta implementación no solo refuerza el dominio del lenguaje ensamblador y la arquitectura MIPS, sino que también introduce al estudiante en el diseño orientado a hardware, donde se deben considerar restricciones como el tiempo de respuesta, la gestión de memoria y la compatibilidad con periféricos. El proyecto sienta así las bases para desarrollos más complejos y escalables en entornos embebidos y sistemas digitales.

## 10. Referencias

- [1] Patterson, D. A., & Hennessy, J. L. *Computer Organization and Design MIPS Edition: The Hardware/Software Interface*, 5ta ed., Morgan Kaufmann, 2014.
- [2] Harris, D. M., & Harris, S. L. *Digital Design and Computer Architecture*, 2da ed., Morgan Kaufmann, 2012.
- [3] Digilent Inc., *Spartan-6 FPGA Development Board Reference Manual*, 2011. Disponible en: <https://reference.digilentinc.com>
- [4] Rodríguez, R., *Diseño de videojuegos clásicos en lenguaje ensamblador MIPS*, Universidad Nacional de Ingeniería, 2021.